

Centro de Investigación y de Estudios Avanzados
del Instituto Politécnico Nacional
Unidad Guadalajara

Auto-diseño de algoritmos en el marco de Sistemas de Auto-Ingeniería (SES)

Tesis que presenta:

Miguel Angel Salazar Martinez

para obtener el grado de:

Maestro en Ciencias

en la especialidad de:

Ingeniería Eléctrica

Director de Tesis

Dr. Mario Angel Siller González Pico

Guadalajara, Jalisco

Agosto de 2026

Auto-diseño de algoritmos en el marco de Sistemas de Auto-Ingeniería (SES)

Tesis de Maestría en Ciencias Ingeniería Eléctrica

Por:

Miguel Angel Salazar Martinez

Ingeniero en Sistemas Computacionales
Instituto Tecnológico de Ocotlán 2019-2023

Becario de SECIHTI, expediente no. xxxxxxxx

Director de Tesis

Dr. Mario Angel Siller González Pico

Resumen

Abstract

Dedicatoria

A mi madre por ser mi ejemplo a seguir. Agradezco infinitamente tu amor y apoyo incondicional. A mi hermana Saidy y mi hermano Diego, por impulsarme a seguir adelante y ser mi espacio seguro. A mi abuela por enseñarme lo que es ser un buen ser humano. A la vida, por permitirme llegar tan lejos.

Agradecimientos

A todos los que me apoyaron en el camino y me ayudaron en esta travesía del posgrado.
A mi asesor de tesis, el Dr. Mario Siller, por todo el apoyo durante el desarrollo del plan de estudios.

A mi querida familia, a mi madre, a mi padre, a mis hermanos, a mis abuelos, a mis tios y primos que tanto me han apoyado.

Al grupo de redes de CINVESTAV Guadalajara, que más que grupo, es una familia.

A los doctores del área de computación, por compartir su conocimiento, consejos y, sobre todo, por cultivar la tenacidad. Esa que no aparece en la rubrica, pero que resulta indispensable para llegar hasta aquí.

A todos mis amigos (incluyendo los compis y roomies) especialmente a los miembros del laboratorio de computación, por esas largas charlas, orientación, momentos de calidez y de recreación.

A todos mis maestros, quienes me educaron en toda mi vida académica y nunca desistieron al enseñarme.

Entre ellos destaco a la profesora Aurora Berenice Navarro Nuñez, por darme el apoyo que necesite al momento de ingresar a este posgrado.

A todos aquellos que creyeron en mí.

Finalmente, quiero agradecer a la SECIHTI por el apoyo económico brindado, con el cual se logró concluir esta maestría.

Muchas Gracias a todos.

Quiero aclarar que no nombré a todos los involucrados, no por falta de gratitud, sino porque en extensión, los agradecimientos serian inmensos.

Índice general

Resumen	I
Abstract	II
Dedicatoria	III
Agradecimientos	IV
1. Introducción	1
1.1. Planteamiento del Problema	2
1.2. Motivación	3
1.3. Objetivos	3
1.3.1. Objetivo General	3
1.3.2. Objetivos Específicos	3
1.4. Hipótesis	4
1.5. Estructura del documento de la Tesis	4
2. Marco Teórico	6
2.1. Sistemas de Auto-Ingeniería	6
2.1.0.1. Fases de Operación y Mantenimiento (O & M) y Evolución	6
2.1.0.2. Actividades de Self-O & M y Self-Evolution	7
2.1.0.3. Punto de Ruptura / Breaking Point	7
2.1.0.4. Elasticidad del Sistema / System Elasticity	7
2.1.0.5. Metamorfosis / Metamorphosis	7
2.1.0.6. Gestión de Cambios de Requerimientos / Requirements Change Management (RCM)	8
2.1.1. Meta-Arquitectura del Marco de Sistemas de Auto-Ingeniería	8
2.2. Actividades de diseño	9

2.2.1.	Diseño de Arquitectura	9
2.2.2.	Diseño de Interfaces	9
2.2.3.	Diseño de Componentes	9
2.2.4.	Diseño de Estructuras de datos	9
2.2.5.	Diseño de Algoritmos	10
2.3.	Sistemas multiagentes MAS	10
2.3.1.	Fundamentos y arquitecturas de los Sistemas Multiagentes	10
2.3.2.	Tipología cognitiva de agentes: Modelos de caja negra (puramente estadísticos/reactivos) vs. Modelos de caja gris (híbridos o neuro-simbólicos)	10
2.3.3.	Paradigmas de coordinación y comunicación inter-agente	10
2.4.	Modelos grande de lenguaje (LLM)	10
2.4.1.	Principios de inferencia y generación de texto en modelos de lenguaje	11
2.4.2.	Capacidades de los LLMs en la comprensión, diseño y generación de código	11
2.4.3.	Limitaciones actuales: Alucinaciones, ventanas de contexto y deficiencias en el razonamiento algorítmico determinista	11
2.5.	Arbol de busqueda de Monte-Carlo (MCTS)	12
2.5.1.	Espacios de búsqueda y métodos de toma de decisiones secuenciales	12
2.5.2.	Fases algorítmicas del MCTS	12
2.6.	Ontología como representacion de conocimiento	12
2.6.1.	Definición y estructura de las ontologías computacionales (clases, atributos, relaciones)	13
2.6.2.	Lenguajes de modelado ontológico	13
2.6.3.	Modelado semántico para la taxonomía y clasificación de problemas algorítmicos	13
3.	Estado del Arte	14
3.1.	Conceptualizacion del Auto-diseño	14
3.2.	Automatización del Diseño	15
3.2.1.	Autodiseño basado en Inteligencia Artificial	15
3.2.2.	Autodiseño bio-inspirado	15
3.2.3.	Autodiseño basado en modelado de agentes	15
3.2.4.	Autodiseño basado en el reuso y busqueda	15
3.3.	Conceptos emergentes similares al auto-diseño	15

3.4. Comparacion de trabajos	15
4. Propuesta	16
4.1. Metodologia	16
4.1.1. Revision Sistemática de literatura	16
4.1.1.1. Formulación preguntas de investigación	16
4.1.1.2. Estrategia de busqueda	17
4.1.1.3. Criterios de Inclusion y de exclusion	21
4.1.1.4. Proceso de seleccion de estudios	23
4.1.1.5. Extracción de datos	25
4.1.1.6. Resultados	26
4.1.1.7. Discusión	26
4.1.2. Definicion y conceptualizacion	26
4.1.3. Desarrollo de la metodologia Prometheus	27
4.1.3.1. Especificacion del sistema	27
4.1.3.2. Diseño Arquitectonico	27
4.1.3.3. Diseño Detallado	27
4.1.4. Analisis y validacion	27
4.2. Propuesta marco conceptual y arquitectura	27
4.2.1. Definicion conceptual y formal	27
4.2.2. Propuesta detallada	27
4.2.3. Resumen Conceptual	27
5. Experimentación y Análisis de Resultados	28
5.1. Análisis de selección automática de algoritmos	28
5.1.1. Análisis de la arquitectura: auto-diseño	28
5.2. Experimentación	28
5.2.1. Caso de estudio: nuevo problema, nueva solución	28
5.2.2. Experimento: Identificación del Problema	28
5.2.3. Experimento: Identificación del Algoritmo	28
6. Conclusiones y Trabajo Futuro	29
6.1. Conclusiones	29
6.2. Objetivos logrados	29
6.3. Trabajo a futuro	29

Referencias Bibliográficas	30
A. Código Fuente Relevante	33

Índice de figuras

4.1. Review process flow chart based on PRISMA.	24
4.2. Proceso de seleccion de articulos con resultados cuantitativos por etapa de cribado.	25

Índice de tablas

4.1. Actividades de diseño y sus correspondientes artefactos de diseño.	17
4.2. Example of Complete Search Query (Scopus)	19
4.3. Resumen de las consultas en Scopus (Obviando los filtros temporales y de dominio para mantener la brevedad).	20
4.4. Resumen de los criterios de inclusion y exclusion	22

Capítulo 1

Introducción

El desarrollo de sistemas de software contemporáneos está intrínsecamente ligado a la evolución de la Ingeniería de Software. A lo largo de las últimas décadas, se ha evidenciado que las actividades de mantenimiento, actualización y evolución de artefactos de software representan una etapa con alto coste en términos de capital humano, tiempo de desarrollo y recursos financieros [1]. Para mitigar esta dependencia, el paradigma de los Sistemas de Auto-Ingeniería (*Self-Engineering Systems* - SES) introduce un marco conceptual y una meta-arquitectura que habilitan a los sistemas de software para ejecutar modificaciones autónomas sobre sí mismos, abriendo la posibilidad de realizar procesos de auto-evolución o metamorfosis [2]. **El propósito fundamental de un sistema SES es gestionar y ejecutar su propio proceso de ingeniería más allá de su ciclo de vida convencional, prescindiendo de la intervención humana.** El ciclo de vida del software comprende el marco integral de evolución del sistema desde su concepción hasta su retiro, estructurado formalmente a través de procesos técnicos que abarcan el análisis de negocio o misión, la definición de necesidades de los interesados y de requisitos del sistema, la definición de la arquitectura y el diseño, el análisis del sistema, la implementación, la integración, la verificación, la transición, la validación, la operación, el mantenimiento y el desecho final del sistema [3].

Dentro de este ciclo autónomo, una de las actividades fundamentales y de mayor impacto es la ingeniería de diseño. Esta fase es crítica debido a que es la encargada de traducir los requerimientos abstractos del entorno en especificaciones técnicas detalladas, estructuras de datos, interfaces y asignación de componentes indispensables para la posterior implementación y ejecución del sistema. En el contexto de los sistemas SES, automatizar esta etapa —es decir, dotar al sistema de capacidades de auto-diseño— implica que la

plataforma no solo reaccione a fallos, sino que sea capaz de reconfigurar conceptualmente su propia lógica operacional de manera válida, segura y coherente con las decisiones arquitectónicas previas.

Sin embargo, replicar la totalidad del ciclo de ingeniería de diseño constituye un desafío multidimensional, con esto referimos al hecho de balancear costos, sostenibilidad, seguridad, resiliencia, etc. [4] [5]. Por ello, y considerando las delimitaciones temporales propias de una investigación de posgrado, este trabajo se acota estrictamente a trabajar con cajas negras y únicamente explicar la actividad del *auto-diseño de algoritmos*. Bajo este alcance, la resolución de dicha actividad se aborda mediante un enfoque formal de asociación problema-solución. Este mecanismo se operativiza a través de una especificación ontológica que proporciona una taxonomía estructurada de problemas, permitiendo al sistema clasificar un requerimiento dinámico y determinar la solución algorítmica idónea para su posterior implementación autónoma.

1.1. Planteamiento del Problema

El diseño de software es una etapa crucial en el ciclo de desarrollo que abarca desde las abstracciones de alto nivel representadas formalmente en las distintas vistas de la arquitectura de software [6] hasta la especificación granular de sus componentes. Tradicionalmente, esta actividad demanda un consumo elevado de recursos organizacionales y un esfuerzo humano significativo. Aunque las metodologías han evolucionado hacia el paradigma de la Ingeniería de Software 3.0 donde los sistemas basados en agentes autónomos han ganado terreno [7], los sistemas actuales carecen de mecanismos para modificar de forma autónoma su lógica algorítmica interna sin alterar o comprometer las estructuras arquitectónicas preexistentes.

Los sistemas no poseen mecanismos de diseño que contemplen un proceso articulado para cada actividad del diseño, sobre todo descripciones de caja blanca.

El problema técnico radica en la ausencia de un mecanismo capaz de acoplar un agente diseñador de algoritmos con las actividades de diseño previas (arquitectura, componentes y estructuras de datos). Al no existir un modelo de abstracción que trate a estas capas previas como una entidad de tipo caja negra, los sistemas adaptativos no pueden actualizar su lógica operacional de manera aislada y segura ante cambios dinámicos en sus requerimientos. Para resolver esta limitación, se requiere un modelo donde la ontología actúe como el puente semántico entre el espacio del problema (los nuevos requerimien-

tos clasificados taxonómicamente) y el espacio de la solución (los algoritmos disponibles), permitiendo que la selección e integración ocurran de forma autónoma y sin romper las interfaces preexistentes.

1.2. Motivación

La motivación central de esta investigación radica en la necesidad de desarrollar sistemas de software con un alto grado de autonomía, capaces de autodiseñar su propia lógica interna cuando las demandas del entorno lo requieran. Al automatizar la generación de artefactos de diseño y la selección de algoritmos mediante el uso de ontologías y agentes de diseño, se establece un habilitador crítico para la evolución y metamorfosis de los sistemas SES [2]. Este enfoque no solo reduce la dependencia del factor humano en entornos críticos y dinámicos, sino que optimiza los costos operativos asociados al mantenimiento evolutivo del software, transformando la adaptación heurística en un proceso de inferencia lógica y clasificación formal.

1.3. Objetivos

1.3.1. Objetivo General

Desarrollar un marco de trabajo y su respectivo flujo para el diseño automatizado en sistemas de auto-ingeniería, el cual sea capaz de diseñar soluciones para posteriormente actualizar o implementar artefactos ante cambios en los requerimientos, abstrayendo las actividades de diseño (arquitectura, componentes, interfaces, estructuras de datos...) mediante un modelo de interacción de agentes de tipo caja negra con un agente diseñador de caja blanca.

1.3.2. Objetivos Específicos

- Sintetizar el estado del arte en materia de diseño de software automatizado y selección algorítmica autónoma, mediante la ejecución y análisis de una Revisión Sistemática de la Literatura (RSL), con el fin de identificar las tendencias, limitaciones y brechas de conocimiento existentes.
- Caracterizar los mecanismos de evolución e implementación autónoma de algoritmos,

analizando los criterios técnicos y las restricciones operacionales requeridas para su ejecución sin intervención humana.

- Diseñar una meta-arquitectura que gobierne la gestión, sustitución e integración de cambios algorítmicos, definiendo explícitamente las interfaces de comunicación tipo caja negra entre las estructuras de diseño preexistentes y el agente diseñador propuesto.
- Validar la viabilidad y efectividad del marco de trabajo desarrollado a través del despliegue de casos de estudio en escenarios experimentales controlados, evaluando su capacidad de adaptación frente a la variabilidad de requerimientos.

1.4. Hipótesis

Es posible dotar a un sistema de un mecanismo que le permita analizar cambios en sus requerimientos y gestionar de manera autónoma el rediseño de sí mismo, ya sea mediante la generación de un nuevo diseño o la adaptación del diseño existente cuando este continúe satisfaciendo los nuevos requerimientos.

1.5. Estructura del documento de la Tesis

La estructura de este documento se organiza en seis capítulos interconectados que guían el desarrollo de la investigación.

El **Capítulo 1** establece la base del estudio mediante la introducción del problema, la motivación, los objetivos, las hipótesis y el contexto general. El **Capítulo 2** expone los fundamentos teóricos indispensables para la comprensión del trabajo, abordando conceptualizaciones clave sobre Sistemas de Auto-Ingeniería (SES, por sus siglas en inglés), automatización, diseño de software y selección algorítmica. Posteriormente, el **Capítulo 3** ofrece una revisión y análisis crítico del estado del arte, identificando las tendencias actuales, los enfoques metodológicos existentes, así como sus limitaciones y brechas de conocimiento. En el **Capítulo 4** se detalla la propuesta central de esta investigación, especificando su metodología, conceptualización, arquitectura y los mecanismos de selección automática diseñados. La validación experimental de dicha propuesta se describe en el **Capítulo 5** a través de casos de estudio, la simulación de escenarios y el análisis riguroso de los resultados obtenidos. Finalmente, el **Capítulo 6** cierra el documento exponiendo las

conclusiones generales, las contribuciones principales de la investigación, sus limitaciones intrínsecas y las directrices sugeridas para el trabajo futuro.

Capítulo 2

Marco Teórico

El Marco Teórico constituye el cimiento conceptual sobre el cual se erige esta investigación. Su propósito fundamental es articular las teorías, paradigmas y modelos previos que otorgan validez científica a la problemática planteada, sirviendo como mapa orientador para el desarrollo de la propuesta.

2.1. Sistemas de Auto-Ingeniería

En [2] se aborda el hecho de que los mismos sistemas no terminen en desecho si no, que sin intervención humana continúen su operación mediante la evolucionando o en su defecto sufran una metamorfosis dando pie a un nuevo sistema, esta perspectiva se conoce como el marco de los sistemas de auto-ingeniería. Los SES contemplan un proceso muy similar a la ingeniería de software tradicional, es decir, las actividades como lo son ingeniería de requerimientos, diseño, implementación, pruebas, etc.[8] tienen lugar en este marco de trabajo. Para este trabajo únicamente nos enfocamos en el diseño del software [3], esto quiere decir que nos vamos a dedicar a entregar artefactos de diseño, nuestros planos del sistema antes de implementar en código.

La teoría SES introduce conceptos que permiten identificar fases y actividades dentro de la vida de estos sistemas [1]:

2.1.0.1. Fases de Operación y Mantenimiento (O & M) y Evolución

A diferencia de la literatura tradicional que debate si el mantenimiento es un subconjunto de la evolución o viceversa, el marco SES establece las fases de Operación y Mantenimiento (O&M) y Evolución como fases independientes.

- Fase de O&M: Se ejecutan mantenimientos correctivos, preventivos, adaptativos, aditivos y de mejora.
- Evolución por Mantenimiento: Se genera cuando los mantenimientos adaptativos, aditivos o perfectivos provocan cambios en los requerimientos.
- Evolución por Cambio de Problema: Ocurre cuando cambia el problema que el sistema busca resolver, siempre y cuando la arquitectura actual sea capaz de soportar dichos cambios.

2.1.0.2. Actividades de Self-O & M y Self-Evolution

Son las actividades fundamentales que el sistema ejecuta de manera autónoma tras su despliegue dentro de las fases de O&M y Evolución, respectivamente.

2.1.0.3. Punto de Ruptura / Breaking Point

Basado en la analogía de los puntos de inflexión (tipping points) en sistemas complejos, representa el umbral en el cual la arquitectura original del sistema ya no puede satisfacer los nuevos requerimientos. Al alcanzar el **Breaking Point**, se suspenden las actividades de Self-O&M y Self-Evolution, desencadenando el inicio de un nuevo proceso e inicio de ciclo de vida de ingeniería.

2.1.0.4. Elasticidad del Sistema / System Elasticity

Fundamentada en los conceptos de evolucionabilidad (evolvability) y resiliencia. Se define como la capacidad del sistema para asimilar requerimientos nuevos o modificados sin la necesidad de construir un nuevo sistema.

2.1.0.5. Metamorfosis / Metamorphosis

Es la fase fundamental del paradigma SES, definida como la **trascendencia del sistema original (Sistema A) hacia un nuevo sistema (Sistema B)** mediante un nuevo proceso de Auto-Ingeniería tras alcanzar un punto de ruptura. Involucra la ejecución coordinada de las actividades SES *Self-**: **Self-Requirements*, *Self-Design*, *Self-Implementation*, *Self-Testing** y **Self-Deployment*.

Se desencadena bajo dos condiciones principales:

- Cuando se requiere un cambio de arquitectura a nivel conceptual (el problema o dominio de aplicación cambió y la arquitectura actual no puede resolverlo).
- Cuando los cambios de requerimientos impactan la arquitectura lógica y esta es incapaz de acomodarlos.

A diferencia de la ingeniería tradicional donde un arquitecto humano toma la decisión, en el marco SES el propio sistema toma la decisión de manera autónoma.

2.1.0.6. Gestión de Cambios de Requerimientos / Requirements Change Management (RCM)

Estructura de requerimientos funcionales: Se dividen en dos componentes principales: **Funcionalidad** (las acciones principales expresadas en verbos) y **Configuración** (parámetros de tiempo, modo o condición). En los requerimientos no funcionales, las métricas y parámetros se tratan como componentes de configuración. Impacto según el componente: Si el cambio ocurre solo en la configuración, el sistema lo absorbe dentro de sus fases de O&M y evolución. Si el cambio afecta la funcionalidad o genera un impacto en la arquitectura que la actual no soporta, se provoca una metamorfosis. Proceso RCM adaptado para SES (basado en [8]):

- *Especificación del Cambio*: Recepción y validación de la solicitud de cambio.
- *Análisis del Cambio*: Evaluación de la elasticidad del sistema para determinar si el cambio produce una nueva versión o si se alcanzó el Breaking Point.
- *Implementación del Cambio*: Si la arquitectura lo soporta, el sistema evoluciona a una nueva versión; si no lo soporta, se inicia un nuevo proceso de Auto-Ingeniería dando lugar a un nuevo sistema (metamorfosis).

2.1.1. Meta-Arquitectura del Marco de Sistemas de Auto-Ingeniería

Los SES cuentan con el ether, el centro de operación de estos sistemas, recordando el hecho de la existencia de dos momentos clave, la evolución y la metamorfosis. Un sistema SES busca gestionar y ejecutar su propio proceso de ingeniería más allá de su ciclo de vida convencional, prescindiendo de la intervención humana. el ether se encarga de primeramente el mantenimiento el cual corregir errores y defectos,

Los SES manejan la evolución como una fase independiente a la de operación y mantenimiento en el ciclo de vida del software, los mantenimientos correctivos, preventivos, adaptativos, aditivos y de mejora **ISOIEEE** pueden dar lugar a la fase de evolución.

2.2. Actividades de diseño

Las actividades principales de la ingeniería de diseño son:

1. Diseño de Arquitectura
2. Diseño de Interfaces
3. Diseño de componentes
4. Diseño de Estructuras de datos
5. Diseño de Algoritmos

2.2.1. Diseño de Arquitectura

Esta especificación según [8] es la parte en la cual se hace una visión general del sistema y como debe operar.

2.2.2. Diseño de Interfaces

Esta actividad especifica como es que las interfaces van a operar antes de incluso diseñar componentes, una vez declarado esto los componentes solo adaptan estas maneras de comunicación para garantizar cohesión y acoplamiento requerido.

2.2.3. Diseño de Componentes

Aquí la actividad especifica como cada componente opera, cumpliendo funciones y objetivos específicos.

2.2.4. Diseño de Estructuras de datos

Esta parte funciona para modelar todas las estructuras de datos que se usarán en los algoritmos, además de proporcionar diferentes esquemas para las bases de datos y

viceversa, es decir se pueden moldear tablas a partir de estructura de datos necesarias o diseñar estructuras de datos a partir de los requerimientos de información en las bases de datos.

2.2.5. Diseño de Algoritmos

Esta es la actividad en la que indagaremos, se encarga del diseño de algoritmos, según la necesidad, eficiencia y eficacia de lo que necesitemos resolver.

2.3. Sistemas multiagentes MAS

Los sistemas multiagentes abordan un paradigma donde cada agente puede ser de distinto tipo, reactivos, BDI, basados en lógica, basados en planeación, basados en utilidad, basados en aprendizajes, de pizarra negra, basados en mercado, o híbridos mezclando características entre los mismos.

2.3.1. Fundamentos y arquitecturas de los Sistemas Multiagentes

Un agente se puede caracterizar por cualquier elemento que tenga una función.

2.3.2. Tipología cognitiva de agentes: Modelos de caja negra (puramente estadísticos/reactivos) vs. Modelos de caja gris (híbridos o neuro-simbólicos)

2.3.3. Paradigmas de coordinación y comunicación inter-agente

2.4. Modelos grande de lenguaje (LLM)

Los LLM son tecnologías que procesan el lenguaje natural, derivado de un entrenamiento previo del modelo, estas son grandes herramientas que por sí solas ayudan mucho a tareas de escritura y de análisis derivado de los patrones que ya reconocen.

2.4.1. Principios de inferencia y generación de texto en modelos de lenguaje

Los Modelos son capaces de inferir cierta informacion basado en el contexto dado en los prompts, y su entrenamiento, por eso, puede responder segun reglas de escritura a lo que se ponga de entrada, dado esto, funcionan como una caja negra, tenemos una entrada un prompt y la salida de dicho prompt, la respuesta varia de modelo a modelo, por el entrenamiento, la ventana de contexto y la calidad de la informacion.

2.4.2. Capacidades de los LLMs en la comprensión, diseño y generación de código

Como tal el contexto dado al LLM rige que puede dar como salida, en el caso de la ingenieria de software, el modelo debe de estar entrenado con mucha informacion formal y de campo del area de dominio, para de esa manera reconocer patrones y replicarlos dado el input de la funcion, en el caso de la ingenieria de diseño, dar la especificacion de los patrones de diseño, y poder compara con su propio entrenamiento que plantillas se ajustan mas a lo requerido, es decir, si nos piden una web sencilla con interacciones, quizas lo mejor sea un comprtamiento monolitico, pero bajo el entrenamiento del LLM nos recomiende mejor un enfoque de MVC (Modelo, Vista, Controlador).

2.4.3. Limitaciones actuales: Alucinaciones, ventanas de contexto y deficiencias en el razonamiento algorítmico determinista

Existen varios contras de los LLM, el primero con las alucinaciones, el segundo es la falta de contexto, esto puedo derivar en malas respuestas, conocidas como alucinaciones, supongamos que si o si los LLM generan una respuesta entonces retornan lo mas cercano.^a la respuesta correcta segun su entrenamiento y el contexto dado, por eso aveces aunque no sea una respuesta correcta, contestan algo aproximado y aveces si es que su modelo no tiene informacion relevante, si divaga y contesta algo que no tiene nada en comun, eso es una alucinacion, ahora bien en ventanas de contexto, tenemos 2 tipos, de entrenamiento y de tokens, los tokens son los valores dados a ciertas cadenas de texto, estas nos permiten dar importancia a lo escrito, por ejemplo la mujer tiene el pelo verde, con tres palabras relevantes podemos sacar lo mas valioso del la frase, mujer, pelo y verde, con eso sabemos que es lo que queremos comunicar.

2.5. Arbol de busqueda de Monte-Carlo (MCTS)

Un Monte Carlo Tree Search, (MCTS) es un algoritmo de búsqueda y toma de decisiones que construye un árbol de forma incremental usando simulaciones aleatorias para evaluar opciones y elegir la acción más prometedora, cada nodo es un estado o configuraciones de un problema y las aristas son transiciones (acciones) de nodo a nodo [9].

Suele repetirse en cuatro fases: selección, expansión, simulación y propagacion hacia atras (retropropagacion).

- **Selección:** se baja por el árbol eligiendo nodos “buenos” según una regla como UCB/UCT, que equilibra exploración y explotación.
- **Expansión:** se añade uno o más nodos hijos para representar nuevas acciones posibles.
- **Simulación:** se hacen jugadas o recorridos aleatorios (o guiados) desde ese nodo para estimar su valor.
- **Retropropagacion:** el resultado de la simulación se propaga hacia arriba para actualizar visitas y valores de los nodos anteriores.

2.5.1. Espacios de búsqueda y métodos de toma de decisiones secuenciales

2.5.2. Fases algorítmicas del MCTS

El arbol de busqueda de montecarlo nos permite identificar patrones en un conjunto de soluciones, nos permite converger en una entre mas iteraciones tengamos.

2.6. Ontología como representacion de conocimiento

Una ontología es una especificación formal y explícita de un dominio: define conceptos, relaciones, individuos, axiomas y restricciones para que el conocimiento pueda ser compartido, reutilizado y procesado por máquinas [10]. Se caracterizan por representaciones semánticas porque no solo organiza datos, sino que hace explícito el significado de esos datos y las relaciones entre entidades, lo que permite inferencia, interoperabilidad y comprensión automática por software.

2.6.1. Definición y estructura de las ontologías computacionales (clases, atributos, relaciones)

2.6.2. Lenguajes de modelado ontológico

Aquí va OWL

2.6.3. Modelado semántico para la taxonomía y clasificación de problemas algorítmicos

El trabajo de [11] une estos temas, para lograr la selección automática de algoritmos, esto en sistemas del internet de las cosas (IoT), la ontología deriva del trabajo [12] donde se propone una taxonomía para la toma de decisiones bajo incertidumbre, considera características como el Objetivo, Tipo de Procesamiento, Capa de procesamiento, Interacción, incertidumbre, representación de conocimiento y un enfoque algorítmico.

Capítulo 3

Estado del Arte

Este capítulo presenta y analiza lo que existe en la literatura científica acerca del diseño automatizado o auto-diseño. Incluyen definiciones, técnicas y marcos de trabajo,

3.1. Conceptualización del Auto-diseño

Sin lugar a dudas el trabajo de [13] es el trabajo mas cercano al auto-diseño, concibe al Self-Designing Software como al propio software convertido en un miembro activo de su equipo de diseño. La tecnica principal es el "Hot-Swapping", este intercambio en caliente permite intercambiar componentes de software en un sistema vivo de forma segura, con el uso de "building blocks", estos bloques de construccion estan validados y cuentan con interfaces especificas para su acople o desecho.

Sin embargo no cubre el diseño como un conjunto de actividades que nos entregan artefactos de diseño [8] es asi que en una busqueda mas a profundidad mediante una revision sistematica de literatura, encontramos actividades total o parcialmente automatizadas que buscan satisfacer el diseño automatizado.

El trabajo de [14] describe una metodología para automatizar el diseño de estructuras de datos a través de la síntesis de estructuras personalizadas. En lugar de depender de diseños tradicionales fijos, este enfoque construye estructuras a la medida partiendo de un conjunto de primitivas de diseño fundamentales lógicas y físicas clasificadas en su "Tabla Periódica de las Estructuras de Datos" [15]. El conjunto combinatorio de estas primitivas conforma un vasto "espacio de diseño" en el cual se evalúan rigurosamente las restricciones de rendimiento y utilización de recursos. Tras este análisis, el diseño óptimo es enviado a un generador de código para sintetizar el diseño en código fuente

ejecutable para el sistema final. De igual manera (en la etapa de diseño de estructuras de datos) la herramienta COZY[16] y mejorada en [17], presentan una solución muy parecida a la mencionada previamente, la entrada de esta herramienta consta de el conjunto de instrucciones que describan que tipo de datos lógicos (usuario, grupos, membresías), que se busca en esos datos y que operaciones implican (actualizar, borrar, etc), para así llegar a la estructura óptima. Esto se lleva a cabo a través de un proceso iterativo, puede llegar a crear estructuras intermedias que no son las mejores, así que se evalúa la eficiencia y eficacia de las mismas, tiene un mecanismo de verificación y garantías, mediante inferencia busca contraejemplos (apoyado por el resolutor SMT Z3) y por último un implementador de código.

3.2. Automatización del Diseño

3.2.1. Autodiseño basado en Inteligencia Artificial

3.2.2. Autodiseño bio-inspirado

3.2.3. Autodiseño basado en modelado de agentes

El trabajo de [18] propone un

3.2.4. Autodiseño basado en el reuso y búsqueda

3.3. Conceptos emergentes similares al auto-diseño

3.4. Comparación de trabajos

Capítulo 4

Propuesta

4.1. Metodologia

La metodologia consta de dos etapas principales, la primera que fue realizar una revision sistematica de literatura sobre tecnicas, marcos de trabajo y demas propuestas que llevaran a cabo el autodiseño, esta misma se llevo a cabo siguiendo la metodologi prisma, los resultados obtenido ayudaron a fundamentar la toma de decision para construir la propuesta. La siguiente etapa fue la construccion de un sistema multiagentes, se compararon diferentes metodologias, pero se decanto por prometheus ya que implementa los conceptos BDI de manera natural, la comparacion con metodologias esta dada con (Gaia, Tropos, Mase, O-mase, Adelfe, Pass)

4.1.1. Revision Sistematica de literatura

El primer paso para desarrollar y fundamentar mi propuesta fue realizar un revision sistematica de literatura, en la que siguiendo los lineamientos PRISMA [19], con este proceso estructurado se realizo la revision de manera exhaustiva, el objetivo principal de esta Systematic Literature Review (SLR), es conocer como se ha automatizado el diseño de sistemas, conocer el estado del arte permite reconocer vacios en el mismo.

4.1.1.1. Formulación preguntas de investigación

La formulacion de una pregunta o preguntas de investigación en cualquier investigación es la piedra angular de la misma, en una SLR no es la excepción, para esto se utilizo el marco de trabajo PICO [20], La Poblacion (P), habarca los procesos de diseño de softwa-

re, desde las tareas tradicionales automatizadas, hasta el auto-diseño bajo el marco de los sistemas de autoingeniería (SES) o que tengan relación directa con ellos. La intervención (I) se refiere a todos los enfoques automatizados, autónomos o impulsados por IA dentro de la vasta literatura, para apoyar a dichas actividades. Estos enfoques comprenden modelos, técnicas y mecanismos destinados a permitir que los sistemas generen, evalúen, o evolucionen artefactos de diseño de manera autónoma. La Comparación (C), implica un análisis comparativo entre las metodologías o técnicas convencionales, contra enfoque progresivamente automatizados. Finalmente, el resultado previsto, denominado Resultado (Outcome en inglés), destaca en una síntesis sistemática del conocimiento existente para mapear esta evolución tecnológica, evaluar las teorías e identificar las brechas de investigación pendientes que obstaculizan la realización de un verdadero auto-diseño.

Con base a esta formulación las siguientes preguntas son propuestas:

- **RQ1:** ¿De qué maneras se han automatizado o hecho autónomos los procesos tradicionales de ingeniería de diseño de software?
- **RQ2:** ¿Qué términos o conceptos emergentes son equivalentes o cercanos a una actividad de diseño totalmente autónoma?
- **RQ3:** ¿Cómo se ha abordado el **autodiseño** de sistemas en el contexto de los SES?

4.1.1.2. Estrategia de búsqueda

Para contestar a la primera pregunta de investigación (RQ1), el análisis fue estructurado alrededor de las actividades en el diseño de software, considerando su correspondencia con enfoques automáticos o autónomos, las etapas se muestran en la tabla 4.1.

Tabla 4.1: Actividades de diseño y sus correspondientes artefactos de diseño.

Actividades de diseño	Artefactos de diseño
Diseño de Arquitectura	Arquitectura del sistema
Diseño de interfaz	Especificación de interfaces
Diseño de componentes	Especificación de componentes
Diseño de estructuras de datos	Especificación de estructuras de datos
Diseño de algoritmos	Especificación de algoritmos

Esta descomposición facilita evaluar en qué medida cada fase del diseño ha sido reinterpretada, respaldada o automatizada bajo los principios de automatización de autogestión.

Esto ofrece una vision amplia del diseño en nuestra busqueda. De tal manera que una busqueda granular se consigue con sub-preguntas de investigacion, al final se contesta la RQ1.

- **SRQ1.1** ¿De qué manera se ha conceptualizado, implementado o evaluado el auto-diseño arquitectónico o la automatización del diseño arquitectónico en la literatura?
- **SRQ1.2** ¿Cómo se ha generado o abordado la noción de autodiseño de interfaces o de diseño automatizado de interfaces en la investigación existente?
- **SRQ1.3** ¿En qué medida se ha explorado y tratado el autodiseño de componentes o el diseño automatizado de componentes en estudios previos?
- **SRQ1.4** ¿Cómo se han definido e investigado en la literatura los enfoques sobre el autodiseño de estructuras de datos o el diseño automatizado de estructuras de datos?
- **SRQ1.5** ¿Cómo se ha abordado el autodiseño de algoritmos o el diseño automatizado de algoritmos?

Para proveer de una respuesta mas comprensiva a la segunda pregunta de investigacion (RQ2), se realizo una busqueda exploratoria de manera exploratoria, usando terminologia reciente y emergente. Este paso apunta a identificar conceptos e investigaciones que de una manera no explicita busca el auto-diseño. como resultado tenemos topicos como diseño evolucionario de sistemas de software, diseño generativo en software, automatizacion del diseño de software e inteligencia artificial para ingenieria de software (AI4SE), estos topicos se añadieron a la estrategia de busqueda. Por ultimo, la manera de abordar la tercer pregunta de investigacion (RQ3), se adopto una estrategia dirigida por el concepto clave de auto-diseño, algo que de manera explicita nos dirigiera a esto. así que se suman los siguiente terminos: Auto-diseño, Sistemas de Auto-Arquitectura y Sistemas autosuficientes.

Las busquedas de la SLR fue realizada en cuatro bases de datos de alto impacto en el campo de la ingenieria de software: Scopus, Web of Science, IEEE Xplore y ACM Digital Library. Estas plataformas se seleccionaron debido a la disponibilidad de acceso institucional completo y a su amplia cobertura de publicaciones revisadas por pares, incluidos artículos de revistas, actas de conferencias y capítulos de libros relevantes para la inteligencia artificial, la ingeniería de software y los sistemas autónomos y autonómicos. Google Scholar si bien es un buen motor de busqueda, no es una base de datos de manera directa y presenta limitaciones en la replicabilidad de las busquedas, por lo tanto no fue una fuente

primaria de búsqueda [21], sin embargo en el proceso de bola de nieve (snowballing) si se uso para la búsqueda de las citas, tanto de entrada como de salida.

Las estrategias de búsqueda se elaboraron sistemáticamente combinando términos relacionados con el tema mediante operadores booleanos (AND, OR) para estructurar explícitamente sus relaciones lógicas. La búsqueda se restringió al campo TITLE-ABS-KEY con el fin de aumentar la precisión conceptual y reducir el ruido en la recuperación de información. Posteriormente, se aplicaron criterios de inclusión y exclusión predefinidos para refinar y validar el conjunto de estudios resultante. Un ejemplo de una consulta completa esta escrita en la tabla 4.2.

Tabla 4.2: Example of Complete Search Query (Scopus)

Database	Scopus
Search Field	TITLE-ABS-KEY
Time Range	2015–2026
Subject Areas	Computer Science (COMP), Engineering (ENGI)
Complete Query	TITLE-ABS-KEY (((("self-architecting" OR "self architectural" OR " auto-architecting" OR "auto architectural") W/3 "system*") OR (("self-architectural" OR "auto-architectural") W/3 "design"))) AND PUBYEAR > 2014 AND PUBYEAR < 2027 AND (LIMIT-TO (SUBJAREA, "COMP") OR LIMIT-TO (SUBJAREA, "ENGI"))

Las consultas presentadas a continuacion corresponden a un conjunto resumido de expresiones que inicialmente fueron ejecutadas en Scopues, y posteriormente adaptadas a las sintaxis especificas de las diferentes bases de datos, se listan las consultas en la tabla 4.3.

Tabla 4.3: Resumen de las consultas en Scopues (Obviando los filtros temporales y de dominio para mantener la brevedad).

Topic/filter name	Summarized Query (Field: TITLE-ABS-KEY)
1. Self-Architecting Systems	("self-architecting system*" OR "auto-architecting system*" OR "self-architectural design" OR "auto-architectural design")
2. Self-Design	("self-design" OR "self design" OR "auto-design" OR "auto design" OR "self software design" OR "auto software design")
3. Evolutionary Design in Software Systems	((("evolutionary design" OR "evolutionary software design") AND ("software system*" OR "software engineering"))
4. Generative Design SW Systems	((("generative design" OR "generative software design") AND ("software system*" OR "software engineering"))
5. Software Design Automation	("software design automation" OR "design automation in software engineering" OR "automated software design" OR "AI for software design")
6. Self-Sufficient Systems	("self-sufficient system*" OR "autonomous self-sufficient software" OR "self-sufficient software system*")
7. AI for Software Engineering	((("AI for software engineering" OR "artificial intelligence in software engineering" OR "machine learning in software engineering"))
8. Self/Auto-Architectural Design	("self-architectural design" OR "auto-architectural design" OR "self adaptive architecture" OR "self-adaptive design")
9. Self/Auto-Interface Design	("self-interface design" OR "auto-interface design" OR "autonomous interface design" OR "adaptive interface design")
10. Self/Auto-Component Design	("self-component design" OR "auto-component design" OR "autonomous component design" OR "self-generated component*")
11. Self/Auto-Data S. Design	("self-data structure design" OR "auto-data structure design" OR "autonomous data structure" OR "self-optimizing data structure*")
12. Self/Auto-Algorithm Design	("self-algorithm design" OR "auto-algorithm design" OR "autonomous algorithm" OR "automated algorithm" OR "generative algorithm design" OR "AI-based algorithm")

Las consultas fueron ejecutadas de manera independiente y posteriormente los resultados fueron unificados (eliminando duplicados). El conjunto final de artículos formaron los *estudios iniciales seleccionados* de esta revisión, recordando que fueron filtrados mediante los criterios de inclusión y exclusión descritos a continuación.

4.1.1.3. Criterios de Inclusión y de exclusión

Para garantizar la calidad y relevancia de los estudios seleccionados, se establecen criterios explícitos de inclusión y exclusión de acuerdo al objetivo de esta revisión, la cual se centra en la automatización del diseño en el marco de los SES.

Criterios de inclusión

- **CI1: Periodo Temporal.** publicaciones desde **2015 hasta Marzo de 2026**. Un rango inicial (2005 – 2025) fue considerado, pero se actualizó para buscar técnicas y avances en el área de la Inteligencia Artificial, evadiendo la literatura del *computo autonomico* para enfocarnos únicamente en los sistemas de autoingeniería.
- **CI2: Idioma.** Solo artículos en **Ingles** fueron incluidos para alinearnos con el lenguaje más adoptado en la divulgación científica en el mundo.
- **CI3: Disponibilidad y Claridad.** los estudios deben proveer **acceso completo** y una **metodología claramente definida** para permitir un análisis comprensivo y total de los mismos.
- **CI4: Dominio específico.** Únicamente investigaciones en el campo de la **Ingeniería de software** que aborde de manera explícita o de manera indirecta cubra lo que es el auto-diseño de una manera unificada como proceso total o parcial.
- **CI5: Relevancia Científica.** Selección de estudios que han demostrado impacto en la comunidad científica a través de las citas, mientras se busca balancear este criterio, evaluando publicaciones recientes que no tienen un acumulado de citas pero sí una gran contribución teórica o técnica en el tema.

Criterios de exclusión

Los siguientes criterios fueron usados para filtrar artículos que no coincidían con los requerimientos de la investigación:

- **CE1: Duplicación.** publicaciones duplicadas o múltiples versiones del mismo estudio; en cuyos casos, solo el más completo o versión más reciente será considerada.

- **CE2: Tipo de Documento. Documentos incompletos, resúmenes extendidos, o publicaciones de posters** que carecen de suficiente detalle técnico para su evaluación.
- **EC3: Irrelevancia Disciplinaria.** estudios fuera del dominio de **Ingeniería de Software** sin un enlace directo al auto-diseño.
- **CE4: Deficiencias Metodológicas.** estudios que carezcan de **rigor metodológico**, particularmente lo que no tienen una clara descripción de las técnicas propuestas, algoritmos o conceptos.

Estos filtros (revisar Tabla 4.4) permiten la identificación de estudios representativos y de alta calidad, centrados en avances recientes en el ámbito conceptual y prácticos en el área de auto-diseño y auto-ingeniería, garantizando así la validez y relevancia de esta revisión sistemática de la literatura (este capítulo en particular).

Tabla 4.4: Resumen de los criterios de inclusión y exclusión

Tipo	Criterio
Inclusion	CI1: Publicaciones entre 2015 y 2025. CI2: Publicaciones en Inglés. CI3: Disponibilidad completa con metodología detallada. CI4: Centrado únicamente en software engineering, self-engineering, or self-design. CI5: Estudios relevantes con contribuciones recientes o número de citas destacables.
Exclusion	CE1: Publicaciones duplicadas o múltiples versiones del mismo estudio. CE2: Resúmenes, posters o documentos incompletos. CE3: Estudios fuera del dominio de los sistemas de software. CE4: Trabajos con falta de detalles en su metodología o rigor científico.

4.1.1.4. Proceso de selección de estudios

Este bloque describe como fue la selección de estudios siguiendo las pautas de PRISMA. La Figura 4.1 ilustra el flujo general de identificación de estudios, cribado (aplicación de criterios de inclusión y exclusión), evaluación de elegibilidad (lectura del resumen y, posteriormente, del documento completo) e inclusión final. Los artículos semilla (conjunto inicial) se seleccionaron a partir de los estudios que cumplían los criterios de inclusión y obtuvieron las puntuaciones más altas en la evaluación de calidad durante la búsqueda inicial en bases de datos. Posteriormente, se identificaron estudios adicionales mediante la técnica de bola de nieve (hacia atrás y hacia adelante), siguiendo las directrices propuestas por [22]. Este proceso iterativo consistió en examinar las listas de referencias de los artículos semilla (bola de nieve hacia atrás) e identificar los artículos que los citaban (bola de nieve hacia adelante), aplicando los mismos criterios de inclusión y exclusión en cada iteración.

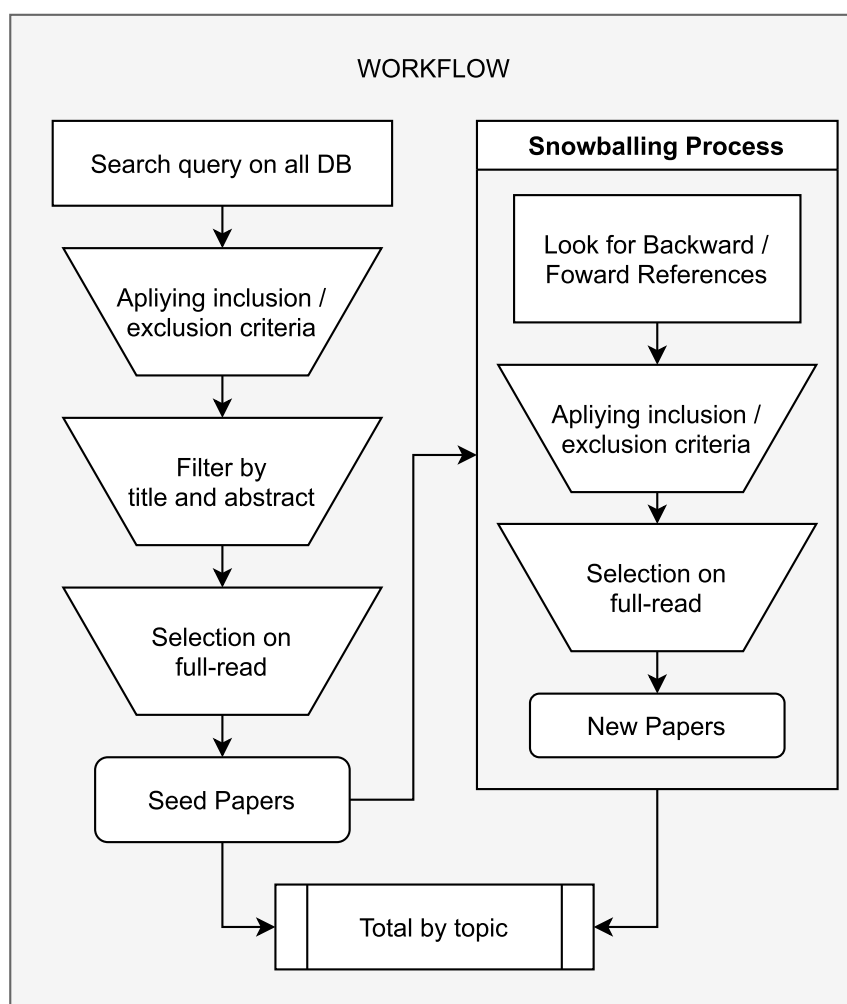


Figura 4.1: Review process flow chart based on PRISMA.

Para complementar el flujo de PRISMA, se añade una vista preeliminar a manera de cuantificar los resultados, así detallar los los estudios seleccionados por cada etapa del proceso y especificando el topico en particular. Esta representación permite comprender mejor cómo el conjunto inicial de estudios recuperados se refinó progresivamente hasta llegar a los estudios seleccionados finalmente y analizados en esta revisión. La Figura 4.2 resume este proceso mediante la especificacion explicita del umero de articulos por cada etapa de seleccion.

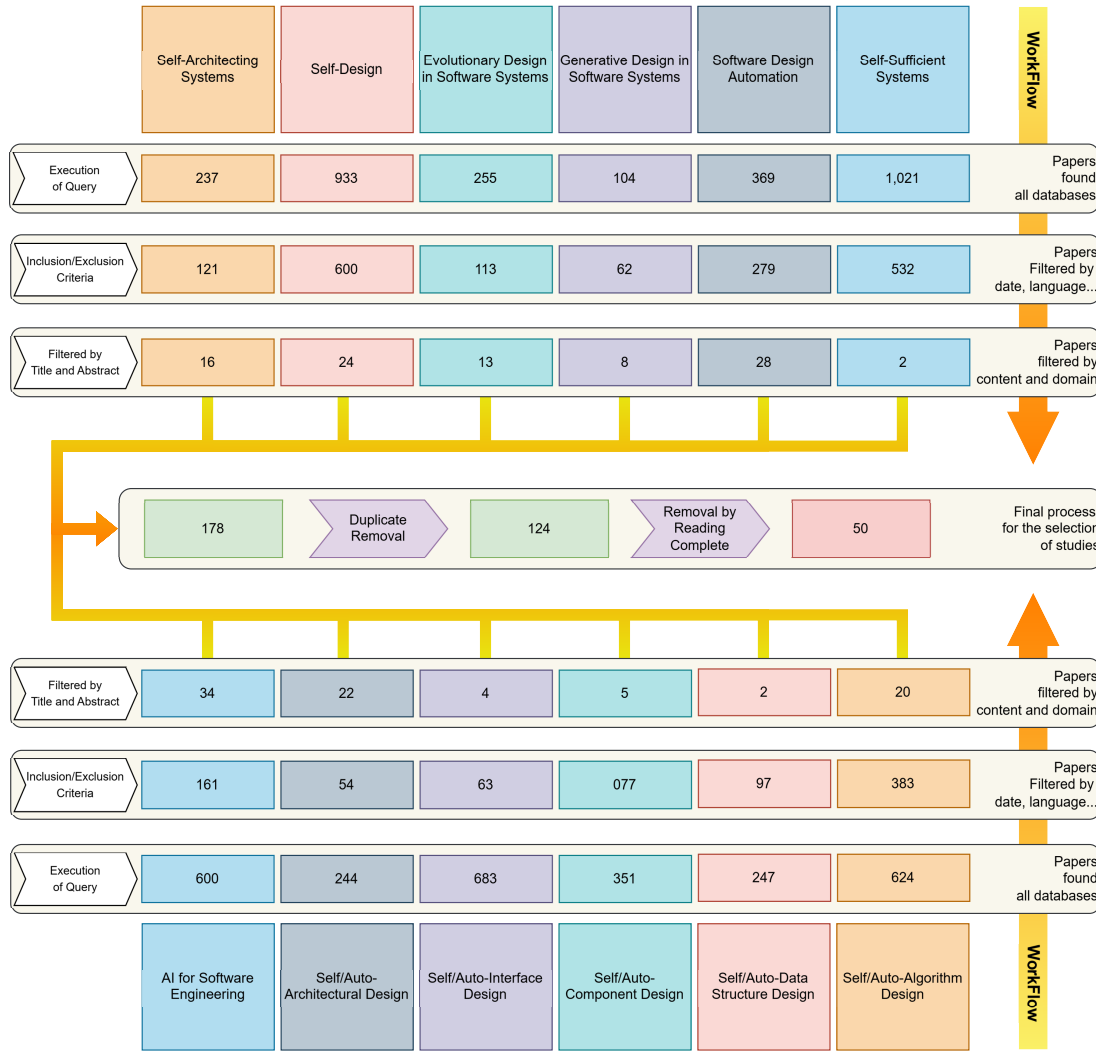


Figura 4.2: Proceso de seleccion de articulos con resultados cuantitativos por etapa de cribado.

4.1.1.5. Extracción de datos

La extracción de datos se realizó exclusivamente a partir de los estudios que cumplían con los criterios de inclusión definidos y presentados en la Tabla 4.4. Este proceso se llevó a cabo de manera sistemática para garantizar la coherencia y la trazabilidad de la información recopilada de cada estudio seleccionado. Dada la heterogeneidad de la literatura analizada, se empleó un análisis narrativo para sintetizar la información. En este contexto, los enfoques analizados se organizaron en categorías temáticas derivadas de los objetivos de la revisión y de las preguntas de investigación formuladas. Esta estrategia permitió identificar patrones recurrentes, tendencias emergentes y áreas poco exploradas dentro de

los estudios seleccionados. Asimismo, la estructuración temática facilitó una evaluación comparativa de cómo cada estudio aborda el autodiseño en el contexto de la ingeniería de software, destacando las similitudes y diferencias entre los enfoques propuestos, así como el grado en que sus hallazgos contribuyen a responder a las preguntas de investigación planteadas en esta revisión.

4.1.1.6. Resultados

Los resultados completos están en la publicación titulada *Automated Software Design Towards Zero Human Intervention: A Systematic Literature Review*; pero para este trabajo están implementados en el Marco Teórico y mayormente en Estado del Arte.

4.1.1.7. Discusión

La siguiente es una síntesis de lo que fue la discusión completa.

La revisión revela un punto de inflexión hacia la automatización del diseño de software a través de cuatro paradigmas principales: la refactorización basada en búsqueda (SBR), el autodiseño algorítmico mediante mejora genética (GI), la adaptabilidad formal con ontologías y planificación (PDDL), y la transición hacia la Ingeniería de Software 3.0 (SE 3.0) basada en ecosistemas multiagente. Aunque estos enfoques permiten manipular arquitecturas complejas y redefinen el rol del desarrollador hacia el de un orquestador estratégico, enfrentan importantes desafíos técnicos: la pérdida de semántica en los modelos matemáticos, la inestabilidad durante el reemplazo de código en caliente, la sobrecarga computacional en entornos inciertos y el riesgo de alucinaciones de diseño ante la falta de restricciones verificables y gobernanza en los agentes. Finalmente, se evidencia una brecha crítica entre el potencial teórico y la práctica, ya que la mayoría de los estudios se limitan a entornos controlados o sintéticos; por ello, se concluye que la integración del razonamiento simbólico con modelos de lenguaje es indispensable para lograr arquitecturas autogeneradas transparentes, funcionales y viables en entornos de producción reales.

4.1.2. Definición y conceptualización

El auto-diseño se define como el conjunto de tareas que un sistema realiza para dar como resultado artefactos de diseño, especificaciones claras y un documento que cubra íntegramente lo que en el futuro será el software especificado por los requerimientos, ya sea en caso de elasticidad (operación y mantenimiento o evolución) o de metamorfosis.

4.1.3. Desarrollo de la metodología Prometheus

Para dar solución a este problema, se empleó la metodología de prometheus, para poder diseñar y brindar un sistema multiagente con un flujo de trabajo determinado y una descripción de caja blanca del agente de algoritmos, evidentemente lo ideal es que todos los agentes sean de caja blanca pero esta fuera del alcance de este trabajo. [23]

Prometheus cubre tres fases:

- 1. La fase de especificación del sistema se centra en identificar los objetivos y las funcionalidades básicas del sistema, junto con las entradas (percepciones) y las salidas (acciones).
- 2. La fase de diseño arquitectónico utiliza los resultados de la fase anterior para determinar qué tipos de agentes incluirá el sistema y cómo interactuarán entre sí.
- 3. La fase de diseño detallado analiza el funcionamiento interno de cada agente y cómo llevará a cabo sus tareas dentro del sistema en general.

4.1.3.1. Especificación del sistema

4.1.3.2. Diseño Arquitectónico

4.1.3.3. Diseño Detallado

4.1.4. Análisis y validación

4.2. Propuesta marco conceptual y arquitectura

4.2.1. Definición conceptual y formal

4.2.2. Propuesta detallada

4.2.3. Resumen Conceptual

El sistema se estructura como un sistema multiagente orientado al proceso de diseño de software. Los agentes colaboran mediante un flujo de trabajo secuencial, en el cual cada agente posee una responsabilidad de diseño específica y produce información que condiciona las decisiones de los agentes posteriores. La metodología Prometheus se emplea para identificar y modelar los agentes, sus metas, percepciones, acciones, capacidades e interacciones.

Capítulo 5

Experimentación y Análisis de Resultados

5.1. Análisis de selección automática de algoritmos

5.1.1. Análisis de la arquitectura: auto-diseño

5.2. Experimentación

5.2.1. Caso de estudio: nuevo problema, nueva solución

5.2.2. Experimento: Identificación del Problema

5.2.3. Experimento: Identificación del Algoritmo

Capítulo 6

Conclusiones y Trabajo Futuro

6.1. Conclusiones

6.2. Objetivos logrados

6.3. Trabajo a futuro

Referencias Bibliográficas

- [1] G. Pérez-Carrillo, M. Siller y L. Alonso, «Self-Engineering Systems: A New Conceptual Framework and a Reference Meta-Architecture for its Life Cycle Management,» *Automated Software Engineering*, mayo de 2026. DOI: [10.21203/rs.3.rs-9577141/v1](https://doi.org/10.21203/rs.3.rs-9577141/v1) dirección: <http://dx.doi.org/10.21203/rs.3.rs-9577141/v1>
- [2] G. P. Pérez Carrillo, «Nuevo Marco Conceptual de Sistemas de Autoingeniería y una Arquitectura para la Gestión de su Ciclo de Vida,» Tesis de Maestría en Ciencias, Centro de Investigación y de Estudios Avanzados del Instituto Politécnico Nacional Unidad Guadalajara, Guadalajara, México, ago. de 2024.
- [3] IEEE Computer Society, *Guide to the Software Engineering Body of Knowledge (SWEBOK)*, 4.^a ed. Los Alamitos, CA: IEEE, 2025, ISBN: 978-0-7695-5166-1. dirección: <https://www.computer.org/education/bodies-of-knowledge/software-engineering>
- [4] A. Maier, J. Oehmen y P. E. Vermaas, «Engineering Systems Design: A Look to the Future,» en *Handbook of Engineering Systems Design*. Springer International Publishing, 2022, págs. 1035-1046, ISBN: 9783030811594. DOI: [10.1007/978-3-030-81159-4_31](https://doi.org/10.1007/978-3-030-81159-4_31) dirección: http://dx.doi.org/10.1007/978-3-030-81159-4_31
- [5] J. A. Crowder, J. N. Carbone y R. Demijohn, *Multidisciplinary Systems Engineering: Architecting the Design Process*. Springer International Publishing, 2016, ISBN: 9783319223988. DOI: [10.1007/978-3-319-22398-8](https://doi.org/10.1007/978-3-319-22398-8) dirección: <http://dx.doi.org/10.1007/978-3-319-22398-8>
- [6] P. B. Kruchten, «The 4+1 View Model of Architecture,» *IEEE Softw.*, vol. 12, págs. 42-50, 1995. DOI: [10.1109/52.469759](https://doi.org/10.1109/52.469759)
- [7] H. Li, H. Zhang y A. E. Hassan, *The Rise of AI Teammates in Software Engineering (SE) 3.0: How Autonomous Coding Agents Are Reshaping Software Engineering*, 2025. arXiv: [2507.15003](https://arxiv.org/abs/2507.15003) [cs.SE]. dirección: <https://arxiv.org/abs/2507.15003>

- [8] I. Sommerville, *Software Engineering*, 9.^a ed. Boston: Addison-Wesley, 2011, ISBN: 978-0-13-703515-1.
- [9] M. Świechowski, K. Godlewski, B. Sawicki y J. Mańdziuk, «Monte Carlo Tree Search: a review of recent modifications and applications,» *Artificial Intelligence Review*, vol. 56, n.º 3, págs. 2497-2562, 2022, ISSN: 1573-7462. DOI: [10.1007/s10462-022-10228-y](https://doi.org/10.1007/s10462-022-10228-y) dirección: <http://dx.doi.org/10.1007/s10462-022-10228-y>
- [10] N. Guarino, D. Oberle y S. Staab, «What Is an Ontology?» En *Handbook on Ontologies*. Springer Berlin Heidelberg, 2009, págs. 1-17, ISBN: 9783540926733. DOI: [10.1007/978-3-540-92673-3_0](https://doi.org/10.1007/978-3-540-92673-3_0) dirección: http://dx.doi.org/10.1007/978-3-540-92673-3_0
- [11] L. Duran Polanco, «Selección de algoritmos para la toma de decisiones en el IoT basada en un modelo semántico,» Tesis de Maestría, Centro de Investigación y de Estudios Avanzados del Instituto Politécnico Nacional, Unidad Guadalajara, Guadalajara, Jalisco, México, feb. de 2026.
- [12] L. Durán-Polanco y M. Siller, «A taxonomy for decision making in IoT systems,» *Internet of Things*, vol. 24, pág. 100904, 2023, ISSN: 2542-6605. DOI: <https://doi.org/10.1016/j.iot.2023.100904> dirección: <https://www.sciencedirect.com/science/article/pii/S2542660523002275>
- [13] B. Porter, P. Faulkner Rainford y R. Rodrigues-Filho, «Self-Designing Software,» *Commun. ACM*, vol. 68, n.º 1, págs. 50-59, dic. de 2024, ISSN: 0001-0782. DOI: [10.1145/3678165](https://doi.org/10.1145/3678165) dirección: <https://doi-org.access.biblioteca.cinvestav.mx/10.1145/3678165>
- [14] S. Idreos et al., «Learning Data Structure Alchemy,» eng, *Bulletin of the IEEE Computer Society Technical Committee on Data Engineering*, vol. 42, págs. 46-57, 2019.
- [15] S. Idreos et al., «The Periodic Table of Data Structures.,» *IEEE Data Eng. Bull.*, vol. 41, n.º 3, págs. 64-75, 2018.
- [16] C. Loncaric, «Data structure synthesis,» en *Proceedings of the 2016 24th ACM SIGSOFT International Symposium on Foundations of Software Engineering*, ép. FSE 2016, Seattle, WA, USA: Association for Computing Machinery, 2016, págs. 1073-1075, ISBN: 9781450342186. DOI: [10.1145/2950290.2983946](https://doi.org/10.1145/2950290.2983946) dirección: <https://doi.org/10.1145/2950290.2983946>

- [17] C. Loncaric, M. D. Ernst y E. Torlak, «Generalized data structure synthesis,» en *Proceedings of the 40th International Conference on Software Engineering*, ép. ICSE '18, Gothenburg, Sweden: Association for Computing Machinery, 2018, págs. 958-968, ISBN: 9781450356381. DOI: [10.1145/3180155.3180211](https://doi.org/10.1145/3180155.3180211) dirección: <https://doi.org/10.1145/3180155.3180211>
- [18] R. Li et al., «Maad: Automate software architecture design through knowledge-driven multi-agent collaboration,» *arXiv preprint arXiv:2507.21382*, 2025.
- [19] M. J. Page et al., «The PRISMA 2020 statement: an updated guideline for reporting systematic reviews,» *BMJ*, vol. 372, 2021. DOI: [10.1136/bmj.n71](https://doi.org/10.1136/bmj.n71) eprint: <https://www.bmj.com/content/372/bmj.n71.full.pdf>. dirección: <https://www.bmj.com/content/372/bmj.n71>
- [20] B. Kitchenham y S. Charters, «Guidelines for performing Systematic Literature Reviews in Software Engineering,» vol. 2, ene. de 2007.
- [21] M. Gusenbauer y N. R. Haddaway, «Which academic search systems are suitable for systematic reviews or meta-analyses? Evaluating retrieval qualities of Google Scholar, PubMed, and 26 other resources,» *Research Synthesis Methods*, vol. 11, n.º 2, págs. 181-217, ene. de 2020, ISSN: 1759-2887. DOI: [10.1002/jrsm.1378](https://doi.org/10.1002/jrsm.1378) dirección: <http://dx.doi.org/10.1002/jrsm.1378>
- [22] C. Wohlin, «Guidelines for snowballing in systematic literature studies and a replication in software engineering,» en *Proceedings of the 18th International Conference on Evaluation and Assessment in Software Engineering*, ép. EASE '14, ACM, mayo de 2014, págs. 1-10. DOI: [10.1145/2601248.2601268](https://doi.org/10.1145/2601248.2601268) dirección: <http://dx.doi.org/10.1145/2601248.2601268>
- [23] L. Padgham y M. Winikoff, «Prometheus: A practical agent-oriented methodology,» en *Agent-oriented methodologies*, IGI Global Scientific Publishing, 2005, págs. 107-135.

Apéndice A

Código Fuente Relevante